

I. Concepts

1. Key Concepts in Cryptography

- **Cryptography:** The science of securing information through secret writing, ensuring secure communication, data, digital signatures, and more.
- **Cryptanalysis:** The science of breaking encrypted texts without knowledge of the key.
- **Cryptology:** A mathematical field studying cryptographic fundamentals.
- **Steganography:** The art of hiding information within a public file (like images or sound), which is **not** cryptography.

2. Cryptographic System

- **Plaintext (M):** The original, unencrypted message.
- **Ciphertext (C):** The encrypted message.
- **Key (K):** Information used for encryption and decryption.
- **Encryption (E()):** The process of converting plaintext to ciphertext.
- **Decryption (D()):** The process of converting ciphertext back to plaintext.

3. Types of Cryptographic Algorithms

- **Symmetric Key Algorithms (1 key):** The same key is used for both encryption and decryption (e.g., AES, DES).
- **Asymmetric Key Algorithms (2 keys):** Different keys for encryption and decryption (e.g., RSA).
- **Stream Ciphers:** Encrypt data one bit at a time.
- **Block Ciphers:** Encrypt data in chunks or blocks.

4. Types of Security

- **Unconditional Security:** Absolute security (e.g., one-time pad) that holds even under future advances in computing power.
- **Computational Security:** Relies on the difficulty of solving certain problems within a reasonable timeframe (e.g., RSA encryption based on factoring large numbers).

5. Vulnerabilities

- **Weak Passwords:** Using easily guessable or default passwords.
- **Lack of Regular Updates:** Failing to update systems and software leaves vulnerabilities open to exploitation.
- **Inadequate Access Control:** Granting excessive privileges allows unauthorized access.
- **Social Engineering:** Manipulating individuals to gain unauthorized access.

- **Insufficient Encryption:** Not encrypting sensitive data, leaving it exposed.
- **Unsecured APIs and Endpoints:** Exposing APIs or endpoints without proper security measures.
- **Poor Incident Response Plans:** Inadequate response to security breaches.
- **Lack of Network Segmentation:** Flat network structure allows lateral movement of attacks.
- **Ignoring Security Logs:** Missing early signs of attacks by not analyzing security logs.
- **Misconfigured Cloud Services:** Exposing sensitive data due to incorrect cloud configurations.
- **Internal Security Neglect:** Overlooking internal security while focusing on perimeter defenses.

6. Solutions for Vulnerabilities

- **Weak Passwords:** Use complex, long passwords and enforce strong policies.
- **Regular Updates:** Implement automated patch management systems.
- **Encryption:** Use strong encryption for data storage and transmission.
- **Social Engineering:** Regular security awareness training.
- **API Security:** Implement proper authentication, authorization, and input validation.
- **Network Segmentation:** Segment networks to limit the spread of attacks.
- **Backup and Recovery:** Regular data backups and tested recovery processes.

7. Categories of Attacks

- **Based on what the attacker can do:**
 - **Passive Attacks:** Attacker gathers information covertly without altering the data/system (e.g., eavesdropping, reconnaissance).
 - **Active Attacks:** Attacker attempts to alter or disrupt operations (e.g., replay attacks, impersonation, tampering).
- **Based on what the attacker knows:**
 - **Ciphertext-only:** Attacker only has the encrypted message.
 - **Known-plaintext:** Attacker knows both the plaintext and its corresponding ciphertext.
 - **Chosen-plaintext:** Attacker can choose the plaintext to be encrypted.
 - **Chosen-ciphertext:** Attacker can choose the ciphertext to be decrypted.
 - **Side-Channel Attacks:** Exploit physical characteristics like power consumption or timing of encryption.

8. Types of Attacks

- **Passive Attacks:**
 - **Eavesdropping:** Intercepting and monitoring communication to gather sensitive information.

- **Reconnaissance:** Gathering publicly available information to find vulnerabilities (e.g., website scanning for email addresses).
- **Passive DNS Monitoring:** Monitoring DNS queries to map out network infrastructure.
- **Active Attacks:**
 - **Replay Attack:** Intercepting and resending valid messages to trick the recipient.
 - **Impersonation:** Pretending to be a legitimate user by intercepting authentication tokens.
 - **Tampering:** Modifying data to cause unauthorized effects.
 - **Denial-of-Service (DoS) / Distributed DoS (DDoS):** Overloading a system or network to make it unavailable to legitimate users.
 - **Injection:** Inserting malicious code into a system or application (e.g., SQL injection).
 - **Man-in-the-Middle (MitM):** Intercepting and potentially modifying communication between two parties.

9. Social Engineering Attacks

- Psychological manipulation to get individuals to disclose confidential information or perform actions:
 - **Phishing:** Trick users into clicking malicious links or revealing information.
 - **Spear Phishing:** Targeted phishing attacks on specific individuals.
 - **Whaling:** Phishing attacks on high-level executives.
 - **Baiting:** Tricking users into downloading malware through tempting ads or infected physical devices.
 - **Smishing:** Phishing through SMS messages.
 - **Business Email Compromise (BEC):** Impersonating a financial executive to make fraudulent wire transfers.

10. Brute Force Attacks

- **Password Cracking:** Systematically trying every possible password.
- **Dictionary Attack:** Trying common words or phrases as passwords.
- **Hybrid Attack:** Combining dictionary attacks with brute force.
- **Credential Stuffing:** Using previously breached usernames and passwords on different systems.

11. Zero-Day Exploits

- **Zero-Day Vulnerability:** An unknown flaw in software that attackers exploit before it is patched.
- **Zero-Knowledge Proof:** A cryptographic method where one party proves to another that a statement is true without revealing any secrets.

- **Zero-Trust Model:** Assumes no entity is trusted automatically; every access request must be verified.

12. Cryptographic System Characteristics

- **Confidentiality:** Hiding the content of a message (e.g., through encryption).
- **Integrity:** Ensuring the accuracy and consistency of data (e.g., using hash functions).
- **Authentication:** Verifying the identities of the parties involved.
- **Non-repudiation:** Ensuring that the sender of a message cannot deny sending it.

13. Mitigation Strategies

- **For Passive Attacks:** Use encryption (e.g., TLS/SSL), network segmentation, IDS, and educate users.
- **For Active Attacks:** Use strong authentication, encryption, intrusion detection/prevention systems, and regular updates.

II. Threat Modelling and Risk assessment

1. Threat Modeling

- **Definition:** Structured approach to identify, quantify, and address security risks in a system.
- **Steps:**
 1. **Identify Assets:** What needs protection (data, apps, networks, etc.)
 2. **Create Architecture Overview:** Diagrams of the system's components.
 3. **Identify Threats:** Use frameworks like **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege).
 4. **Identify Vulnerabilities:** Find weaknesses that could be exploited.
 5. **Assess Risk:** Evaluate the likelihood and impact of threats exploiting vulnerabilities.
 6. **Develop Mitigations:** Define strategies (encryption, firewalls, etc.).
 7. **Validate and Review:** Continuously test and review the effectiveness of mitigations.

2. Risk Assessment

- **Definition:** A systematic process to identify, evaluate, and prioritize risks to an organization's assets.
- **Key Steps:**
 - **Identify Assets:** Understand what needs protection.

- **Identify Threats:** External (hackers) or internal (insiders).
- **Identify Vulnerabilities:** Look for weaknesses in the system.
- **Determine Likelihood:** Estimate the chance of threats exploiting vulnerabilities.
- **Evaluate Impact:** Assess potential damage.
- **Calculate Risk:** Combine likelihood and impact to assess risk.
- **Prioritize Risks:** Focus on the most critical risks.
- **Develop Mitigations:** Define actions to reduce risk.
- **Implement Mitigation:** Apply strategies to lower risk.
- **Monitor and Review:** Continuously check the effectiveness of measures.
- **Risk Assessment Types:**
 - **Qualitative:** Uses descriptive terms (high, medium, low).
 - **Quantitative:** Uses numerical values for a detailed analysis.
 - **Hybrid:** Combines qualitative and quantitative approaches.

3. Methodologies and Frameworks

- **NIST SP 800-30:** Guide for risk assessment.
- **ISO/IEC 27005:** International standard for security risk management.
- **OCTAVE:** Focuses on organizational risk and security planning.
- **TARA (Threat Analysis and Risk Assessment):** Framework for risk management.

4. Common Vulnerabilities and Exposures (CVE)

- **Definition:** A catalog of publicly disclosed security vulnerabilities.
- **Features:**
 - A. Unique CVE IDs for each vulnerability.
 - B. Contains a description, CVSS score, and affected platforms.
 - C. Managed by MITRE Corporation.
- **CVE Process:** Helps in identification, awareness, information sharing, and vulnerability management.
- **CVSS (Common Vulnerability Scoring System):** Standard to measure vulnerability severity but does not measure overall risk.

III. Mathematical background

1. XOR Logical Function

- XOR (exclusive OR) is essential in cryptography, used in systems like OTP (One-Time Pad) ciphers and AES.
- It's the simplest and fastest encryption function, working through mod 2 addition.

- XOR is used in cryptographic systems when the key is randomly generated, has the same size as the message, and is used only once.

2. Modular Arithmetic

- Modular arithmetic is a way to perform calculations with a finite set of values (e.g., modulo n).
- Operations like addition, subtraction, multiplication, and exponentiation can be done in modular space.
- Important in many cryptographic algorithms like RSA.

3. Log₂x

- Logarithms, particularly log base 2 ($\log_2 x$), are important in cryptography for determining the number of bits needed to represent a number in binary.
- Example: $\log_2 35 \approx 5.13$, rounded to 6 bits.

4. Prime Numbers

- Prime numbers play a crucial role in cryptographic systems, such as RSA.
- Methods like the Solovay-Strassen, Fermat, and Rabin-Miller tests help determine whether a number is prime.
- Large prime numbers are especially important for RSA and other cryptographic algorithms.

5. Testing Prime Numbers

- The Rabin-Miller test checks if a number is likely prime by performing modular exponentiation.

6. Integer Factorization Problem

- Integer factorization is the process of breaking down a number into its prime factors and is crucial for cryptographic algorithms such as RSA encryption and signatures.
- The Number Field Sieve (NFS) is the best-known factorization algorithm.

7. Entropy

- Entropy measures the amount of uncertainty or randomness in a set of possible values.
- In cryptography, it's used to quantify how difficult it is to predict a value.
- Higher entropy means more randomness, which is vital for encryption security (e.g., in password generation).

8. Algorithm Complexity

- Cryptographic algorithms are evaluated based on their complexity, including factors like input size and processing time.
- Complexity classes (constant, linear, logarithmic, exponential) categorize problems by how their difficulty scales with input size.

9. P vs NP Problem

- The P vs NP problem asks whether every problem that can be verified quickly (in polynomial time) can also be solved quickly.
- It's a major unsolved problem in computer science.

10. Big Numbers

- Cryptographic systems handle very large numbers (e.g., 1024 or 2048-bit integers), which require specialized techniques for storage and arithmetic operations.
- Multiplication and division with large numbers are complex, often involving optimized algorithms like Karatsuba.

11. Random Numbers

- Random numbers are crucial in cryptography for tasks like key generation and encryption modes.
- There are two types: true random numbers (based on physical processes like radioactive decay) and pseudorandom numbers (generated deterministically by algorithms).
- Entropy is a key measure of randomness, determining the strength of cryptographic keys.

12. True Random Numbers Generators (TRNG)

- **Definition:** Use physical sources that generate unpredictable random events.
- **Sources:**
 - Electronic noise from semiconductor devices
 - Least significant bits from audio channels
 - Interrupt intervals from hardware devices
 - Key press or cursor position logging
- **Processing:** Distilled by cryptographic hash functions to ensure increased bit dependence.
- **Examples:** Cloudflare Lava Lamp wall, Random.org (based on atmospheric noise).

13. Pseudo Random Numbers Generators (PRNG)

- **Definition:** Produce a sequence of values based on a seed and current state. The same seed always generates the same sequence.
- **Types:**

- Statistically random (passes randomness tests).
 - Cryptographically secure (difficult for attackers to predict the next value).
- **Security Strength:** Measured in bits (e.g., 2^n operations needed to break the system).

14. PRNG Characteristics

- Simple and fast.
- Must generate variable-length numbers without repetition.
- Should generate independent and uniformly distributed values.

15. Important PRNG Examples

- **Yarrow:** Open-source PRNG, used in FreeBSD and iOS/macOS for random devices.
- **ANSI X9.17:** Secure PRNG using triple DES (56-bit key) for encryption.
- **Blum Blum Shub Generator (BBS):** Highly secure, uses factorization ($p * q$) to generate pseudo-random numbers.

16. Linear Congruential Generators (LCGs)

- A type of PRNG using a recurrence relation: $X_{n+1} = (aX_n + c) \bmod m$.
- LCGs are **not used in cryptography** anymore due to their predictability.
- Combining LCGs can improve results (e.g., combining modulus 231 and 249).

17. Linear Feedback Shift Register (LFSR)

- Generates pseudo-random sequences by using a feedback sequence (tap sequence).
- Useful in hardware implementation (e.g., GSM encryption using A5).

18. PRNGs Using Cryptographic Methods

- Some PRNGs use repeated encryption of an input for stronger randomness.
- Example: An encryption algorithm generating a pseudo-random number using a 256-period generator.

19. Encoding vs. Encryption

- **Encoding:** The process of converting data into a specific format for efficient storage or transmission (e.g., Base64, ASCII, Unicode).
- **Not encryption:** Encoding does not provide data confidentiality like encryption does.

20. Quantum Computing

- **Challenges:** Not yet practical for large-scale computations due to qubit limitations and environmental noise.

- **Security Threat:** Quantum computing is expected to break current encryption systems like RSA and ECC within a decade.
- **Post-Quantum Cryptography (PQC):**
 - NIST started selecting quantum-resistant algorithms.
 - CRYSTALS-Kyber (public-key encryption) and CRYSTALS-DILITHIUM (digital signature) are among the early winners.
 - In 2024, formal standards for PQC are expected.

21. Protocols

- A series of steps to achieve a specific task between two or more parties.
- **Types:**
 - A. **Arbitrated:** Involves a trusted third party.
 - B. **Adjudicated:** Combines non-arbitrated and arbitrated subprotocols.
 - C. **Self-enforcing:** Does not require third-party arbitration.
- **Use Cases:** Secret key establishment, elections, auctions, and secure multi-party computation.

IV. Hash functions

1. Hash Functions Overview

- A **hash function** is a cryptographic one-way function used to compute a fixed-length hash value for a variable-length message.
- It **does not hide content**, but provides data integrity and is often used for message authentication and digital signatures.

2. Properties of Hash Functions

- **Deterministic:** Same input always produces the same output.
- **Fast Computation:** Efficient for processing large volumes of data.
- **Pre-Image Resistance:** Difficult to find the original message from the hash value.
- **Collision Resistance:** Infeasible for two different messages to produce the same hash value.
- **Puzzle Friendly:** Hard to find an input that generates a specific hash output.
- **Modifying a single bit of input** leads to a drastically different hash.

3. Common Hash Algorithms

- **MD5** (128 bits)
- **SHA-1** (160 bits)

- **SHA-256, SHA-512** (256, 512 bits)
- **SHA-3** (224, 256, 384, 512 bits)
- **RIPEMD-160** (160 bits)

4. Collision Resistance

- Collisions occur when two different inputs produce the same hash. For a hash function h , a collision is when $h(x_1) = h(x_2)$ for $x_1 \neq x_2$. This is hard to find, but not impossible.

5. Birthday Paradox

- **Birthday Attack:** The probability of two inputs having the same hash increases with the square root of the possible input size (for example, with 365 days in a year, you need 23 people for a 50% chance of two sharing a birthday). This concept can be applied to hash functions, reducing the number of attempts needed to find a collision.

6. Pre-Image Resistance and Attacks

- **Pre-image resistance:** Given a hash output, it's difficult to find the original input. This is not impossible, but infeasible for large inputs.
- **Attacks on MD5 and SHA-1:**
 - **MD5:** Collisions found since 2005, making it insecure for certain applications.
 - **SHA-1:** Considered broken in 2005, deprecated in 2011, and attacks were successfully performed in 2017.

7. SHA-3 (Keccak)

- Designed as a new cryptographic hash function that complements SHA-2.
- Uses a **sponge construction**, which makes it suitable for embedded or smart devices. It was chosen in 2012 after a NIST competition.

8. MD5 and SHA-1 Implementation

- Both involve multiple rounds and non-linear functions to compute the hash value. MD5, for example, uses 4 rounds of transformations with specific constants and nonlinear functions (FF, GG, HH, II).

9. Security Considerations

- The effectiveness of hash functions is dependent on their ability to resist pre-image and collision attacks. With advances in computing power, weaker algorithms like MD5 and SHA-1 are no longer considered secure for certain uses, such as digital signatures.

10. Overview of Hash Functions:

- **Usage:**
 - Store passwords securely (convert them to hash values).
 - Validate file integrity (e.g., Git uses SHA-256).
 - Detect modifications (IDS).
 - Extend or reduce secret values to a predefined size.
 - Blockchain uses hash functions as Proof of Work.
 - Used in Version Control Systems to check for changes.

11. Hash Function Characteristics:

- **Not Used For:**
 - Message authentication codes (MACs) are not directly used for securing data.
 - Hiding the initial seed value in a PRNG.

12. Man-in-the-Middle Mitigation:

- **Process:**
 - Alice sends the hash of a message over a secure channel.
 - Bob recomputes the hash and compares it to detect alterations.
 - If the attacker controls the hash transmission, they can alter the message undetected.

13. Intrusion Detection (IDS):

- **Process:**
 - Alice computes and stores hashes for files.
 - If an attacker modifies a file, Alice can compare the new hash to the original.
 - If the hash differs, it signals an alteration.

14. Password Storage & Rainbow Table Attacks:

- **Password Storing Without Salt:**
 - Passwords are stored as hashes (e.g., MD5, SHA-1).
 - Attacker uses a precomputed Rainbow Table to check for common passwords.
- **Password Storing With Salt:**
 - A random salt is added to passwords before hashing, preventing Rainbow Table attacks.
 - Salt value is not secret but helps delay or prevent attacks.
 - Different salts for each user account increase security.
 - **PBKDF2** (Password-Based Key Derivation Function) adds computational effort to slow down attacks.

15. Hashing in Cryptocurrencies:

- **Use in Blockchain:**
 - Hash functions are critical for validating transactions, creating addresses, and ensuring data integrity in blockchains.
 - **Proof of Work (PoW):** Miners find a hash with specific properties.
 - **Proof of Stake (PoS):** Validators are chosen based on stake amount (low energy usage compared to PoW).
 - Merkle Trees are used to verify large sets of transactions efficiently.

16. Forensic and Digital Evidence:

- **Hash-based Tools:**
 - Used to detect file alterations, e.g., Autopsy, HashDeep, Chkrootkit.
 - Helps compare known values with computed ones to spot suspicious activities.

17. Message Authentication Codes (MAC):

- **HMAC:**
 - A keyed hash used for data integrity and authentication.
 - Commonly used in protocols like IPsec and TLS.
 - Requires a secret key known only to the communicating parties.
 - Ensures the message has not been altered during transmission.

18. Practical Examples:

- **Bitcoin Block Hashing:**
 - Involves solving cryptographic puzzles (proof of work) to add blocks to the blockchain.
 - Difficulty adjusts every 2,016 blocks to maintain a consistent rate of block generation (around 10 minutes per block).

V. Symmetric Cryptography Systems

1. Symmetric Cryptographic Systems

- **Types:** Also called conventional, private-key, or single-key encryption.
- **Key Features:**
 - The encryption and decryption keys are identical.
 - Both sender and receiver know the key, which must be protected.
 - Prior to 1970, symmetric encryption was the only form.
 - Most widely used encryption method today.

- Requires a secret key, a strong encryption algorithm, and a secure way to distribute the key.

2. Encryption Algorithms

- Symmetric encryption algorithms rely on two main methods:
 - **Transposition (Permutation)**: Changing the positions of characters.
 - *Example*: Scytale, a tool used for transposition ciphers, was used by the ancient Greeks.
 - Types: Mono-phase, multi-phase, monographic (char-based), multigraphic (group-based).
 - **Substitution**: Replacing characters from the plaintext with characters from a cipher alphabet.
 - *Example*: Caesar Cipher – vulnerable to brute-force.
 - *Variation*: Random substitution cipher increases security.

3. Types of Ciphers

1. **Transposition Ciphers**:
 - The order of characters in the plaintext is altered.
 - Vulnerable to statistical attacks, as the frequency of characters remains unchanged.
 - Examples: Scytale cipher, P-boxes for transposition.
2. **Substitution Ciphers**:
 - Replace each character from plaintext with one from a cipher alphabet.
 - *Caesar Cipher*: A mono-alphabetic substitution cipher, easy to break.
 - *Poly-alphabetic Substitution*: Uses multiple cipher alphabets. Vigenère Cipher is a common example.
 - *Homophonic Substitution*: Multiple replacements for each character to hide frequency patterns.
 - *Playfair Cipher*: Encrypts two letters at a time using a 5x5 matrix. More secure than mono-alphabetic ciphers.
3. **One-Time Pad (OTP)**:
 - A symmetric cipher where the key is as long as the message and used only once.
 - If the key is random and securely distributed, the encryption is unbreakable.
 - However, the major disadvantage is that the key must be as large as the message itself and must be securely transmitted.
4. **Rotor Machines (e.g., Enigma Machine)**:
 - Used in WWII, these machines applied complex, varying substitutions through rotating cylinders, each offering a different substitution.

4. General Considerations

- **Security:** The security of symmetric encryption depends heavily on the secrecy of the key.
- **Implementation:** Symmetric algorithms are often implemented through transposition and substitution methods, with various cipher types available for use.

5. Enigma Machine:

- **Poly-alphabetic Vigenere encryption** implemented by Enigma.
- **Security:**
 - With 3 rotors, there were 17,576 possible solutions.
 - By adding 6 ways to transpose disks, the number of solutions increased to 105,456.
 - The connection table with 10 pairs of characters brought the total to 9.49 million solutions.
- **Design:**
 - Keyboard, electric circuits, rotors with 26 symbols (3–7 rotors), reflector, and a connections table.
 - The settings included rotor order, initial rotor positions, and the initialization of symbols and connections.

6. Lorenz Cipher:

- Used by the German Army during WWII, more advanced than Enigma.
- Bill Tutte broke the Lorenz cipher in 1942 without seeing the machine.
- There were ~200 Lorenz machines with 12 rotors and 501 pins, allowing for 10^{144} different settings.

7. Product Ciphers:

- Composition of multiple functions (substitutions or permutations) to create a more secure encryption.
- **S-P networks** (Substitution-Permutation) are used in modern ciphers and were introduced by Claude Shannon in 1949.
- **Substitution (S-box)** and **Permutation (P-box)** help provide **confusion** (making the relationship between the ciphertext and key complex) and **diffusion** (spreading out plaintext over ciphertext).

8. Feistel Networks:

- Used for block ciphers (such as in the **Lucifer** algorithm).
- A round function f depends on $n/2$ bits for each L and R block, and key scheduling expands the key from N bits to $N * r$ bits for r rounds.
- The Feistel structure allows for easy reversal of encryption during decryption.

9. Modern Block Ciphers:

- Examples include **AES**, **DES**, **Blowfish**, **Twofish**, etc.
- **Block ciphers** work on fixed-size blocks (64 or 128 bits), while **stream ciphers** operate on individual bits or bytes, often using XOR for encryption.
- **Modes of Operation** for block ciphers include **ECB**, **CBC**, **PCBC**, and others.

10. Padding:

- **Padding** is used to adjust the length of the last block in a message.
- Common types: **Zero padding**, **Bit padding**, and **PKCS7**.
- **Ciphertext Stealing (CTS)** can also be an alternative to padding.

11. ECB vs CBC:

- **Electronic Codebook (ECB)**: The same plaintext block always encrypts into the same ciphertext block. It's vulnerable to block replay and does not conceal plaintext patterns.
- **Cipher Block Chaining (CBC)**: Uses a feedback mechanism where the previous ciphertext block influences the encryption of the next block. It is more secure than ECB, but ciphertext errors affect the current and next blocks.

12. Block Cipher Modes:

- **ECB**: Each block encrypted independently. Vulnerable to block replay.
- **CBC**: Feedback from the previous block is used, making it harder to manipulate ciphertext. Requires initialization vector (IV).
- **Key points of CBC**:
 - A ciphertext error affects the corresponding plaintext block and the next block.
 - Enhances security by concealing plaintext patterns.

13. Propagation Cipher Block Chaining (PCBC)

- **Concept**: Both previous plaintext and ciphertext blocks are XORed with the current plaintext block before encryption (or after decryption).
- **Usage**: Used in Kerberos version 4 for encryption and data integrity in one pass.
- **Process**:
 - **Encryption**: $C_i = \text{Encryption}(B_i \text{ XOR } B_{i-1} \text{ XOR } C_{i-1})$
 - **Decryption**: $B_i = \text{Decryption}(C_i) \text{ XOR } B_{i-1} \text{ XOR } C_{i-1}$

14. Cipher FeedBack (CFB)

- **Concept**: Transforms a block cipher into a self-synchronizing stream cipher.
- **Encryption**: $C_i = \text{Encryption}(C_{i-1}) \text{ XOR } B_i$
- **Decryption**: $B_i = \text{Decryption}(C_{i-1}) \text{ XOR } C_i$

- **Advantages:**
 - Conceals plaintext patterns.
 - Input to the block cipher is randomized.
 - Hard to manipulate ciphertext.
- **Disadvantages:**
 - Cannot preprocess data.
 - Ciphertext may be longer than the plaintext.
 - Encryption is not parallelizable.
 - Ciphertext error affects an entire block and next block's corresponding bit.

15. Cipher Output FeedBack (OFB)

- **Concept:** A synchronous stream cipher derived from a block cipher.
- **Encryption:** $C_i = B_i \text{ XOR } E(R_i)$
- **Decryption:** $B_i = C_i \text{ XOR } R_i$
- **Advantages:**
 - Bit errors do not propagate.
 - Vulnerable to message stream modifications.
 - Must never reuse the same sequence (key + IV).
- **Limitation:** Works best with full block feedback (CFB-64 or CFB-128).

16. Counter Mode (CTR)

- **Concept:** A "new" mode similar to OFB, but it encrypts a counter value rather than a feedback value.
- **Encryption:** $C_i = P_i \text{ XOR } O_i$ where $O_i = \text{DESK1}(i)$
- **Decryption:** $B_i = C_i \text{ XOR } R_i$
- **Advantages:**
 - Parallel encryption is possible (good for high-speed networks).
 - Preprocessing is possible.
 - Random access to encrypted blocks.
 - Proven security.
- **Limitation:** Key/counter values must never be reused.

17. Block Cipher Modes Summary:

- **CFB:** Self-synchronizing stream cipher.
- **OFB:** Synchronous stream cipher.
- **CTR:** Turns block cipher into a stream cipher.
- **PCBC:** Used for both encryption and integrity checking.

18. Symmetric Cryptographic Systems (DES, Triple DES, AES):

1. **DES (Data Encryption Standard):**

- 64-bit block cipher with a 56-bit key.
 - Vulnerable to brute-force attacks.
 - Triple DES (3DES) used as a solution, which applies DES thrice with two or three keys for more security.
2. **Triple DES (3DES):**
- Uses either 2 or 3 keys (56-bit each).
 - More secure than DES but slower.
 - Can be less secure than expected, offering around 112-bit security.
3. **AES (Advanced Encryption Standard):**
- Winner of the NIST contest.
 - 128-bit block cipher with 128, 192, or 256-bit keys.
 - Efficient implementation on various CPUs.
 - Stronger and faster than DES and 3DES.

19. Brute-force Attacks:

- Brute-force attack on DES:
 - 56-bit key: 2^{56} possible keys.
 - Modern computing can break it in a few days or months.
- **Time Estimates for Brute-force:**
 - 128-bit key: Takes an enormous amount of time (2^{128} possibilities).

20. Key Size Impact:

- **Larger Key Sizes** (e.g., 128-bit, 168-bit) exponentially increase the difficulty of brute-force attacks.

21. Advanced Encryption Techniques:

- **Multiple Encryption:** Encrypting the same plaintext multiple times with different keys.
- **Cascading:** Using different encryption algorithms together for more security.

22. Key Concepts:

1. **Stream Ciphers:**
 - **Encryption:** Stream ciphers convert plaintext to ciphertext one bit/byte at a time, typically using XOR with a keystream.
 - **Key Generation:** The security of the system depends on the key generator (keystream), which must be long, statistically random, and difficult to predict. A proper design can offer security similar to block ciphers but is often simpler and faster.
 - **One-Time Pad (OTP):** A key that is as long as the message, used only once, and is theoretically unbreakable if the key is kept secure.
2. **RC4 (1987):**

- A widely-used stream cipher with variable key sizes and byte-oriented operation.
- The encryption is fast due to a large key schedule and complex non-linear behavior. However, the key should never be reused.
- Security concerns exist if the key is not properly managed (e.g., with WEP).
- 3. **Types of Stream Ciphers:**
 - **Synchronous:** Keystream is generated independently of the message stream (e.g., OTP).
 - **Self-synchronizing:** Synchronization occurs after an initial random header.
- 4. **Stream Cipher Vulnerabilities:**
 - **Key reuse:** If the same key is used more than once, it can be compromised.
 - **Short key length:** Short keys are vulnerable to brute-force attacks.
 - **Examples:**
 - **Venona Project** (1943-1980) and **MS-PPTP** (Windows NT) demonstrate risks with key management and reuse.
- 5. **RC4 Key Schedule:**
 - The key schedule involves an array of numbers that are shuffled based on the key, generating the keystream used for encryption.
- 6. **Security Issues with WEP and WPA2:**
 - **WEP:** Vulnerable due to the small IV size (24 bits), which leads to key reuse. This allows attackers to break the key in a few minutes.
 - **WPA2:** Susceptible to **KRACK attacks**, which exploit vulnerabilities in the 4-way handshake protocol.
- 7. **Key Management:**
 - The **key length** and its use in cryptographic operations are critical factors for security. For example, a 64-bit key with a small character set can be weak.
 - **Key Generation:** Secure random key generation standards like **ANSI X9.17** should be used to ensure proper security.
 - **Key Storage:** Secure storage is necessary to protect keys, using methods like Hardware Security Modules (HSMs) or Trusted Platform Modules (TPMs).
- 8. **Hardware Security Modules (HSMs):**
 - Specialized devices designed for secure cryptographic operations and key management.
 - HSMs offer high security, performance, and compliance with standards like FIPS 140-2/3, making them critical in sectors like finance and healthcare.
 - Examples of HSMs: Thales Luna, AWS CloudHSM, IBM 4765/4767, Microsoft Azure Dedicated HSM, and Yubico YubiHSM.

23. Summary for Study:

- **Stream ciphers** are fast and efficient but require careful key management to prevent vulnerabilities like key reuse or predictable keys.
- **RC4** is a popular but older stream cipher that's fast but has security concerns, especially with WEP.
- Modern security protocols, like **WPA2**, have their own flaws (KRACK attacks).

- **Key management** and **HSMs** are essential to securely generate, store, and use cryptographic keys in sensitive environments.

VI. Asymmetric Cryptography Systems

1. Overview

- **Asymmetric Cryptographic Systems** (also known as **public key systems**) solved the problem of key distribution in symmetric systems.
- They offer solutions to vulnerabilities such as **key distribution**, **non-repudiation**, and **authentication**.
- It uses a **pair of keys**: a **public key** for encryption and a **private key** for decryption.

2. Key Concepts

- **Public Key Encryption** (introduced by Diffie & Hellman in 1976) enables the use of two distinct but related keys.
- **Public Key** is made available to everyone for encrypting messages, while **Private Key** is kept secret for decryption.
- It's also used for **digital signatures**.

3. Common Algorithms

- **RSA** (Rivest-Shamir-Adleman) – Uses large prime number factorization.
- **El Gamal** – Based on discrete logarithms.
- **Elliptic Curve Cryptography (ECC)** – Uses elliptic curve equations, offering higher security with smaller key sizes.
- **Other Algorithms** include Diffie-Hellman, DSS, LUC, XTR, NTRU, and more.

4. RSA Example (Basic Overview)

- **Key Generation**: Two large prime numbers p and q are selected; $n = p \times q$ and Euler's totient function $\phi(n) = (p-1) \times (q-1)$ is calculated.
- **Encryption**: A message M is encrypted using the public key.
- **Decryption**: The recipient uses their private key to decrypt the message.

5. El Gamal Algorithm

- Based on the discrete logarithm problem.

- **Key Generation:** Select a large prime number p and generator α .
- **Encryption:** Random integer k is selected; ciphertext is calculated as $\gamma = \alpha^k \mod p$ and $\delta = m \times (\alpha^k)^a \mod p$.
- **Decryption:** The message is recovered using the private key a .

6. Elliptic Curve Cryptography (ECC)

- **Elliptic Curves** are defined by equations of the form $y^2 = x^3 + ax + b$.
- **ECC** offers higher security with smaller key sizes compared to RSA.
- **Elliptic Curve Diffie-Hellman (ECDH)**, **Elliptic Curve Integrated Encryption Scheme (ECIES)**, and **Elliptic Curve Digital Signature Algorithm (ECDSA)** are common ECC protocols.

7. Hybrid Encryption Systems

- Combines symmetric (faster) and asymmetric (secure key exchange) encryption methods.
- The **public key** is used to encrypt a **symmetric key**, which is then used for data encryption/decryption.

8. Homomorphic Encryption

- Allows computation on encrypted data without decryption, preserving privacy.
- **Types:**
 - **Partially Homomorphic Encryption (PHE):** Supports only addition or multiplication.
 - **Somewhat Homomorphic Encryption (SHE):** Supports limited operations.
 - **Fully Homomorphic Encryption (FHE):** Supports arbitrary operations on encrypted data.

9. Cryptanalysis

- Cryptanalysis is the art of breaking encryption without the key.
- Attacks include:
 - **Brute-force:** Trying all possible keys.
 - **Cryptanalytic attacks:** Use knowledge of the algorithm to find the key.
 - **Backdoor attacks:** Malicious functions intentionally left in cryptographic systems to allow unauthorized access.

I. Concepts.....	1
1. Key Concepts in Cryptography.....	1
2. Cryptographic System.....	1
3. Types of Cryptographic Algorithms.....	1
4. Types of Security.....	1
5. Vulnerabilities.....	1
6. Solutions for Vulnerabilities.....	2
7. Categories of Attacks.....	2
8. Types of Attacks.....	2
9. Social Engineering Attacks.....	3
10. Brute Force Attacks.....	3
11. Zero-Day Exploits.....	3
12. Cryptographic System Characteristics.....	4
13. Mitigation Strategies.....	4
II. Threat Modelling and Risk assessment.....	4
1. Threat Modeling.....	4
2. Risk Assessment.....	4
3. Methodologies and Frameworks.....	5
4. Common Vulnerabilities and Exposures (CVE).....	5
III. Mathematical background.....	5
1. XOR Logical Function.....	5
2. Modular Arithmetic.....	6
3. Log ₂ x.....	6
4. Prime Numbers.....	6
5. Testing Prime Numbers.....	6
6. Integer Factorization Problem.....	6
7. Entropy.....	6
8. Algorithm Complexity.....	6
9. P vs NP Problem.....	7
10. Big Numbers.....	7
11. Random Numbers.....	7
12. True Random Numbers Generators (TRNG).....	7
13. Pseudo Random Numbers Generators (PRNG).....	7
14. PRNG Characteristics.....	8
15. Important PRNG Examples.....	8
16. Linear Congruential Generators (LCGs).....	8
17. Linear Feedback Shift Register (LFSR).....	8
18. PRNGs Using Cryptographic Methods.....	8
19. Encoding vs. Encryption.....	8
20. Quantum Computing.....	8
21. Protocols.....	9

IV. Hash functions.....	9
1. Hash Functions Overview.....	9
2. Properties of Hash Functions.....	9
3. Common Hash Algorithms.....	9
4. Collision Resistance.....	10
5. Birthday Paradox.....	10
6. Pre-Image Resistance and Attacks.....	10
7. SHA-3 (Keccak).....	10
8. MD5 and SHA-1 Implementation.....	10
9. Security Considerations.....	10
10. Overview of Hash Functions:.....	11
11. Hash Function Characteristics:.....	11
12. Man-in-the-Middle Mitigation:.....	11
13. Intrusion Detection (IDS):.....	11
14. Password Storage & Rainbow Table Attacks:.....	11
15. Hashing in Cryptocurrencies:.....	12
16. Forensic and Digital Evidence:.....	12
17. Message Authentication Codes (MAC):.....	12
18. Practical Examples:.....	12
V. Symmetric Cryptography Systems.....	12
1. Symmetric Cryptographic Systems.....	12
2. Encryption Algorithms.....	13
3. Types of Ciphers.....	13
4. General Considerations.....	13
5. Enigma Machine:.....	14
6. Lorenz Cipher:.....	14
7. Product Ciphers:.....	14
8. Feistel Networks:.....	14
9. Modern Block Ciphers:.....	15
10. Padding:.....	15
11. ECB vs CBC:.....	15
12. Block Cipher Modes:.....	15
13. Propagation Cipher Block Chaining (PCBC).....	15
14. Cipher FeedBack (CFB).....	15
15. Cipher Output FeedBack (OFB).....	16
16. Counter Mode (CTR).....	16
17. Block Cipher Modes Summary:.....	16
18. Symmetric Cryptographic Systems (DES, Triple DES, AES):.....	16
19. Brute-force Attacks:.....	17
20. Key Size Impact:.....	17
21. Advanced Encryption Techniques:.....	17

22. Key Concepts:.....	17
23. Summary for Study:.....	18
VI. Asymmetric Cryptography Systems.....	19
1. Overview.....	19
2. Key Concepts.....	19
3. Common Algorithms.....	19
4. RSA Example (Basic Overview).....	19
5. El Gamal Algorithm.....	19
6. Elliptic Curve Cryptography (ECC).....	20
7. Hybrid Encryption Systems.....	20
8. Homomorphic Encryption.....	20
9. Cryptanalysis.....	20